

SUMO-GEO

Manual de uso

Simulación de movilidad SUMO en el navegador con MapLibre GL JS + deck.gl

Universidad de Cuenca — Ecuador
https://github.com/Pbarbecho/SUMO_GEO

August 25, 2026

Índice

1	Introducción	2
1.1	Arquitectura	2
2	Requisitos previos	2
3	Lanzar una simulación desde cero	2
3.1	Paso 1 — Clonar el repositorio	2
3.2	Paso 2 — Construir y levantar los contenedores	2
3.3	Paso 3 — Abrir la aplicación	3
3.4	Paso 4 — Detener	3
3.5	Volver a lanzar la aplicación (segunda vez y siguientes)	3
4	Comandos de Docker explicados	3
4.1	<code>docker compose up -build</code>	3
4.2	Comandos habituales	4
4.3	Servicios y puertos	4
5	Ubicación de los archivos de mapas de SUMO	4
5.1	<code>mapa/</code> — Red metropolitana (escenario activo por defecto)	4
5.2	<code>sumo/</code> — Centro de Cuenca y demo	5
6	Configuración: variables de entorno del backend	5
6.1	Cambiar de escenario	6
7	Uso de la interfaz	6
7.1	Panel izquierdo (controles; se pliega solo, pasar el cursor para abrir)	6
7.2	Panel derecho: Históricos	6
7.3	Barra superior y cámara	6
7.4	Inspector con clic derecho	7
8	Generación de mapas y demanda (avanzado)	7
9	Solución de problemas	7

1 Introducción

SUMO-GEO es una aplicación web para ejecutar y visualizar simulaciones de movilidad de **SUMO** directamente en el navegador, sin instalar ningún software en el cliente. Ofrece edificios 3D, estimación de tráfico en tiempo real (niveles de servicio A–F), semáforos sincronizados con las fases de SUMO, vehículos 3D procedurales, un inspector con clic derecho y un panel de estadísticas históricas.

1.1 Arquitectura

```
SUMO (Docker) --TraCI--> Backend FastAPI --REST-----> red + edificios + semaforos
(GeoJSON)
(suscripciones)                                --WebSocket--> vehiculos + LOS + fases +
estadísticas                                     --> MapLibre GL JS + deck.gl
(navegador)
```

- **Simulación:** SUMO con TraCI. Cada pestaña del navegador lanza su propia simulación (conexiones TraCI con etiqueta única).
- **Backend:** FastAPI (Python). Convierte la red y los polígonos a GeoJSON, calcula el LOS por arista y transmite los fotogramas por WebSocket.
- **Frontend:** SPA estática con MapLibre GL JS (mapa base y edificios) y deck.gl (calles, vehículos, semáforos).
- **Entrega:** nginx sirve la SPA y hace de proxy inverso de `/api` y `/ws` (mismo origen, sin CORS).

2 Requisitos previos

- **Docker** y **Docker Compose** (Docker Desktop en macOS/Windows, o `docker` + plugin `compose` en Linux).
- **Git** para clonar el repositorio.
- Conexión a internet la primera vez (descarga de imágenes Docker y teselas del mapa base).
- Opcional, solo para *generar mapas nuevos* en el host: Python 3 con `pip3 install eclipse-sumo sumolib`.

No es necesario instalar SUMO en el equipo: el contenedor del backend lo incluye.

3 Lanzar una simulación desde cero

3.1 Paso 1 — Clonar el repositorio

```
git clone https://github.com/Pbarbecho/SUMO_GEO.git
cd SUMO_GEO
```

3.2 Paso 2 — Construir y levantar los contenedores

```
docker compose up --build
```

La primera ejecución tarda unos minutos (construye la imagen del backend e instala SUMO). Las siguientes son inmediatas.

3.3 Paso 3 — Abrir la aplicación

- Frontend: <http://localhost:8082>
- Comprobación del backend: <http://localhost:8000/api/health>

Al abrir la página se conecta el WebSocket, arranca SUMO dentro del contenedor y los vehículos comienzan a circular. Si se cambió el código o la configuración, recargar con *hard refresh*: **Cmd+Shift+R** (macOS) o **Ctrl+Shift+R** (Windows/Linux).

3.4 Paso 4 — Detener

```
docker compose down
```

3.5 Volver a lanzar la aplicación (segunda vez y siguientes)

No hace falta `-build` en cada arranque. La imagen del backend ya está construida y los mapas y el frontend se montan por volumen, así que basta con:

```
docker compose up -d
```

La regla práctica es la siguiente:

Qué cambió desde la última vez	Comando necesario
Nada (solo volver a usar la herramienta)	<code>docker compose up -d</code>
Frontend (<code>frontend/</code> : HTML, JS, CSS)	Nada que reconstruir: recargar el navegador con Cmd/Ctrl+Shift+R
Mapas o demanda (<code>sumo/</code> , <code>mapa/</code>)	<code>docker compose up -d</code> y recargar el navegador (cada conexión lanza un SUMO nuevo que ya lee los archivos actualizados)
Variables <code>APP_*</code> en <code>docker-compose.yml</code>	<code>docker compose up -d</code> (recrea el contenedor con el nuevo entorno)
Código del backend (<code>backend/</code> : Python, <code>requirements.txt</code> , <code>Dockerfile</code>)	<code>docker compose up -d -build</code>

En resumen: `-build` *solo* cuando se toca la carpeta `backend/`. Usarlo de más no rompe nada (Docker aprovecha la caché), solo tarda unos segundos extra.

4 Comandos de Docker explicados

4.1 `docker compose up -build`

Elemento	Significado
<code>docker compose</code>	Orquesta los servicios definidos en <code>docker-compose.yml</code> (aquí: <code>backend</code> y <code>frontend</code>).
<code>up</code>	Crea y arranca los contenedores, redes y volúmenes. Con la terminal abierta muestra los logs de ambos servicios en vivo.

<code>-build</code>	Fuerza la reconstrucción de la imagen del backend antes de arrancar. Necesario tras modificar <code>backend/</code> (código Python, <code>requirements.txt</code> , <code>Dockerfile</code>). No hace falta si solo cambian archivos montados por volumen (<code>frontend</code> , mapas, <code>docker-compose.yml</code>).
<code>-d</code> (opcional)	<i>Detached</i> : arranca en segundo plano y devuelve la terminal. Ejemplo: <code>docker compose up -d -build</code> .

4.2 Comandos habituales

Comando	Uso
<code>docker compose up -d</code>	Arrancar en segundo plano (sin reconstruir).
<code>docker compose down</code>	Parar y eliminar los contenedores.
<code>docker compose restart backend</code>	Reiniciar solo el backend (por ejemplo tras cambiar variables de entorno no requiere <code>-build</code> , pero sí <code>up -d</code> para releer el YAML).
<code>docker compose logs -f</code>	Seguir los logs de todos los servicios (<code>Ctrl+C</code> para salir).
<code>docker compose logs -f backend</code>	Solo los logs del backend: aquí se ven los arranques de SUMO y los errores de TraCI.
<code>docker compose logs -tail=100 backend</code>	Últimas 100 líneas sin seguir.
<code>docker compose ps</code>	Estado de los servicios y puertos publicados.
<code>docker compose build --no-cache backend</code>	Reconstrucción limpia si la imagen quedó en mal estado.

4.3 Servicios y puertos

Servicio	Imagen	Puerto host	Función
backend	construida de <code>backend/Dockerfile</code>	8000	FastAPI + SUMO
frontend	<code>nginx:1.27-alpine</code>	8082	SPA + proxy <code>/api</code> , <code>/ws</code>

5 Ubicación de los archivos de mapas de SUMO

El repositorio incluye dos familias de escenarios. Los directorios del host se montan *de solo lectura* dentro del contenedor del backend:

Directorio (host)	Montaje (contenedor)	Contenido
<code>./sumo</code>	<code>/sumo</code>	Centro de Cuenca (OSM) + demo sintética
<code>./mapa</code>	<code>/mapa</code>	Red metropolitana de Cuenca completa

Por eso las rutas de las variables `APP_*` empiezan por `/sumo/` o `/mapa/`: son rutas *dentro del contenedor*.

5.1 mapa/ — Red metropolitana (escenario activo por defecto)

Archivo	Descripción
---------	-------------

<code>network.net.xml</code>	Red vial de toda Cuenca (~93 MB, 26×19 km, 337 intersecciones semaforizadas). Excluida de git por tamaño; se regenera con <code>scripts/build_city.sh</code> .
<code>metro_low.sumocfg</code> / <code>_mid</code> / <code>_high</code>	Configuración SUMO de cada nivel de tráfico (Bajo / Medio / Alto).
<code>metro_low.rou.xml</code> / <code>_mid</code> / <code>_high</code>	Demanda de 24 h por nivel (6 000 / 15 000 / 30 000 viajes) con perfil horario realista y orígenes/destinos en la componente fuertemente conexas (sin avisos <i>no route</i>).
<code>vtypes.add.xml</code>	Flota de turismos (24 modelos reales: Chevrolet, Toyota, Kia, ...) con dimensiones y masas.
<code>transit.add.xml</code>	Tipo de vehículo bus .

5.2 sumo/ — Centro de Cuenca y demo

Archivo	Descripción
<code>cuenca.net.xml</code>	Red del centro (~1×1 km) importada de OSM, proyectada (se georreferencia exactamente sobre el mapa).
<code>cuenca.poly.xml</code>	Huellas de edificios con altura real (etiquetas OSM) para la extrusión 3D.
<code>cuenca.sumocfg</code>	Escenario demo corto (0–600 s).
<code>cuenca_{low,mid,high}.*</code>	Niveles de 24 h para el centro.
<code>cuenca_bbox.osm.xml</code>	Extracto OSM original de la zona.
<code>grid.net.xml</code> , <code>buildings.poly.xml</code> , <code>demo.sumocfg</code>	Demo sintética (malla 6×6) generada sin internet por <code>scripts/generate_scenario.sh</code> .
<code>vtypes.add.xml</code> , <code>transit.add.xml</code>	Copia de la flota para esta familia.

Para añadir un mapa propio: colocar `.net.xml`, `.poly.xml` (opcional) y `.sumocfg` en `sumo/` (o `mapa/`) y apuntar `APP_SUMO_CONFIG` al `.sumocfg`. Los semáforos, la red y los edificios se derivan automáticamente al arrancar el backend; la red debe construirse con `netconvert -tls.guess-signals` para que existan semáforos.

6 Configuración: variables de entorno del backend

Se definen en la sección `environment` del servicio `backend` en `docker-compose.yml`. Tras cambiar una variable: `docker compose up -d` (releer YAML) y refrescar el navegador.

Variable	Valor por defecto	Significado
<code>APP_SUMO_MODE</code>	<code>managed</code>	managed: el backend lanza SUMO. remote: se conecta a un servidor TraCI externo.
<code>APP_SUMO_CONFIG</code>	<code>/mapa/metro_mid.sumocfg</code>	sumoconfig: de él se leen red, edificios y semáforos al arrancar.

APP_SUMO_CONFIG_TEMPLATE	/mapa/metro_{level}	El nivel del selector Bajo/Medio/Alto; {level} se sustituye por low/mid/high.
APP_SIM_BEGIN	28800	Segundo simulado de arranque (28 800 = 08:00, hora punta).
APP_VIEW_LON / APP_VIEW_LAT	-79.004 / -2.902	Centro inicial de la vista (útil cuando la demanda se concentra en una zona).
APP_NET_FILE	/ —	Sustituyen la red o los edificios del .sumocfg (p. ej. añadir poly a un escenario que no lo lista).
APP_ORIGIN_LON	/ Cuenca	Ancla ENU para redes sintéticas sin proyección (la demo <i>grid</i>).
APP_ORIGIN_LAT		
APP_STEP_LENGTH	1.0	Segundos por paso de simulación.
APP_MAX_FPS	10	Tope de fotogramas por segundo del WebSocket.
APP_SUMO_HOST	/ sumo / 8813	Servidor TraCI en modo remote .
APP_SUMO_PORT		
APP_USE_LIBSUMO	false	Usar libsumo en proceso si está disponible.
APP_CORS_ORIGINS	*	Orígenes CORS permitidos.

6.1 Cambiar de escenario

En `docker-compose.yml` hay un bloque marcado con el escenario *activo* (red metropolitana) y, comentada, la alternativa del centro con edificios 3D. Para cambiar: comentar unas líneas, descomentar las otras y ejecutar `docker compose up -d`.

7 Uso de la interfaz

7.1 Panel izquierdo (controles; se pliega solo, pasar el cursor para abrir)

- **Pausar / Reiniciar** la simulación.
- **Escenario:** nivel de tráfico Bajo / Medio / Alto (reconecta).
- **Velocidad:** 1–30 fps.
- **Edificios, Congestión (LOS), Semáforos, PoI:** capas visibles.
- **Calles concurridas** + umbral $\geq N$ veh: pines flotantes con el conteo en vivo.

7.2 Panel derecho: Históricos

Series temporales con eje de hora simulada: vehículos (autos/buses), CO₂ de la flota, tiempo de viaje medio, espera en semáforos y desglose por tipo.

7.3 Barra superior y cámara

Zoom, órbita automática, alternador 2D/3D y presets de luz (amanecer / día / atardecer / noche). El *pad* **Pan-Tilt** (esquina inferior izquierda) controla giro (horizontal) e inclinación (vertical); doble clic = norte + 3D.

7.4 Inspector con clic derecho

Clic derecho sobre un **vehículo** (tipo, velocidad, CO₂, espera, retraso, distancia, carril, progreso de ruta), una **calle** (carriles, límite, densidad, LOS, ocupación) o un **semáforo** (estado, tipo de soporte). Los turismos de la flota son eléctricos (**evEHICLE**): su CO₂ es 0; los buses sí emiten.

8 Generación de mapas y demanda (avanzado)

En el host, con `pip3 install eclipse-sumo sumolib`:

```
# Importar una ciudad real de OSM (bbox = W,S,E,N):
./scripts/build_city.sh "-79.010,-2.903,-79.000,-2.893" cuenca

# Generar demanda de 24 h en 3 niveles (con duarouter):
python3 scripts/gen_traffic.py \
  --net sumo/cuenca.net.xml --types sumo/vtypes.add.xml \
  --transit sumo/transit.add.xml --outdir sumo --prefix cuenca \
  --low 3000 --mid 8000 --high 18000 --bus-share 0.05

# Variante para redes grandes (sin duarouter, sin avisos "no route"):
python3 scripts/gen_traffic.py ... --no-duarouter --connected
```

Opciones útiles de `gen_traffic.py`: `-center-bbox="W,S,E,N"` (concentrar la demanda en una zona), `-levels low,mid` (regenerar un subconjunto), `-seed` (reproducibilidad).

9 Solución de problemas

Síntoma	Solución
La página muestra “esperando al backend”	El backend aún carga la red (la metropolitana tarda unos segundos) o falló: revisar <code>docker compose logs -f backend</code> .
Cambios de código no se reflejan	Backend: <code>docker compose up -d --build</code> . Frontend: basta <i>hard refresh</i> (Cmd/Ctrl+Shift+R).
“desconectado” permanente	Ver logs del backend; si SUMO no arranca, verificar que <code>APP_SUMO_CONFIG</code> apunta a un <code>.sumocfg</code> existente dentro de <code>/sumo</code> o <code>/mapa</code> .
Puerto 8082 ocupado (<i>port is already allocated</i>)	Ver qué contenedor lo usa con <code>docker ps -a grep 8082</code> , detenerlo (<code>docker stop + docker rm</code>) o cambiar el mapeo "8082:80" del servicio frontend en <code>docker-compose.yml</code> . Cuidado: el backend publica el 8000 — no asignar el mismo puerto a ambos servicios.
Sin semáforos en un mapa propio	La red no los trae: reconstruir con <code>netconvert -tls.guess-signals</code> .